VEB DASTURLAR XAVFSIZLIGI

A9. Veb server, TLS, WAF va xavfsiz konfiguratsiya

1-VARIANT — Apache teskari proksisini TLS va ModSecurity bilan himoyalash

Mashg‘ulot maqsadi. Lokal veb xizmat oldiga Apache teskari proksisini qo‘yish, o‘quv CA orqali domen nomiga mos sertifikat yaratish, TLS va xavfsizlik sarlavhalarini tekshirish, WAFning kuzatish hamda bloklash rejimlarini taqqoslash.

Kutiladigan natija. Talaba sertifikatga ishonch va domen mosligini alohida tekshiradi; HTTPdan HTTPSga yo‘naltirishni, TLS 1.2/1.3 ulanishini va proksi yuboradigan sarlavhalarni kuzatadi. ModSecurity qoidasini On rejimiga o‘tkazib, 403 javobi bilan birga qoida identifikatori hamda backend jurnalini dalil sifatida ko‘rsatadi.

Ko‘rsatkich	Shart
Fan / semestr	Veb dasturlar xavfsizligi / faqat 5-semestr
Mashg‘ulot	A9; 1-variant; individual ish
Auditoriya	30 ta kompyuter; LAN va internet mavjud
Kompyuter	Windows 11; 8 yadroli CPU; RAM 32 GB; SSD 1 TB
Virtual muhit	VMware Workstation yoki VirtualBox; Ubuntu 24.04 LTS
VM resursi	2 vCPU, RAM 4 GB, disk 25 GB; NAT adapter
Vaqt / baho	80 daqiqa, tayyor muhit bilan / 10 ball

Asosiy vositalar

1. Apache HTTP Server 2.4, mod_ssl, mod_headers, mod_proxy va mod_proxy_http.
2. ModSecurity 2.x uchun libapache2-mod-security2 paketi; bitta mahalliy o‘quv qoida.
3. OpenSSL, curl va Python 3; Python backend standart kutubxonada ishlaydi, pip paketi talab qilinmaydi.

Tayyorgarlik. 80 daqiqaga sig‘dirish uchun VM, paketlar, backend.py va httpd.conf oldindan tayyorlanadi. OT o‘rnatish hamda Worddan kod ko‘chirish alohida vaqt oladi. Sertifikat yaratish va himoyani tekshirish darsning o‘zida bajariladi.

Ish chegarasi. Har bir talaba faqat o‘z VMida ishlaydi. Barcha tinglovchilar 127.0.0.1ga bog‘langan; internetga xizmat chiqarilmaydi. Asosiy variant Apache bilan bajariladi. Nginx va IIS bo‘yicha taqqoslash qo‘shimcha topshiriqdir.

Arxitektura va himoya qatlamlari

Bir VMda uchta port ishlatiladi. curl mijozidan HTTPS so‘rovi Apachega boradi; Apache TLSni yakunlaydi, ModSecurity qoidasi orqali so‘rovni tekshiradi va ruxsatli so‘rovni Python backendga uzatadi. Backend tashqi portda tinglamaydi.

Manzil	Vazifa	Kutiladigan holat
http://a9.test:8089	HTTP kirish nuqtasi	301 → https://a9.test:8449/
https://a9.test:8449	TLS + WAF + teskari proksi	200 yoki WAFdan 403
http://127.0.0.1:8009	Python backend	A9-BACKEND markerli javob

a9.test — tajriba uchun tanlangan nom. curl --resolve orqali shu nom 127.0.0.1ga bog‘lanadi: tizim DNS yoki hosts faylini o‘zgartirish shart emas. 30 ta VM bir xil nom va portlardan mustaqil foydalanadi.

Qisqa nazariya

4. TLS transportni himoya qiladi; SQL inyeksiya yoki noto‘g‘ri avtorizatsiyani o‘zi tuzatmaydi. Server sertifikatidagi SAN domen nomiga mos bo‘lishi, zanjir esa mijoz ishonadigan CAga borib taqalishi kerak. [1, 2]
5. Teskari proksi oldindan belgilangan backendga uzatadi. ProxyRequests Off forward proxy rejimini o‘chiradi; ProxyPass xizmat yo‘lini, ProxyPassReverse esa tegishli javob yo‘naltirishlarini boshqaradi. [3]
6. WAF so‘rovga qoidalar tatbiq qiladi. DetectionOnly rejimida moslik qayd etiladi, deny amali bloklamaydi; On rejimida bloklash amalga oshadi. [4]
7. CSP va boshqa javob sarlavhalari brauzerga siyosat beradi. Ularning mavjudligini curl bilan tekshirish mumkin; brauzerda siyosat amalda ishlashini isbotlash esa alohida sinovdir. [5]

Mazkur variant nimani isbotlaydi?

WAF qoidasi faqat probe parametrining A9-WAF-TEST qiymatini taniydi. Bu zararli kod emas, nazorat markeridir. 200 → 403 o‘zgarishi WAFning ayni qoidasi va bloklash rejimi ishlashini ko‘rsatadi; barcha hujumlardan himoya yoki OWASP CRS to‘liq qamrovi isbotlanmaydi.

Izolyatsiya chegarasi. Bir VM ichidagi foydalanuvchi 8009 backendga bevosita murojaat qila oladi. Loopback LANdan kirishni cheklaydi, lekin shu VMdagi jarayonlarni bir-biridan ajratmaydi. Real joylashtirishda backendga kirish tarmoq va xizmat huquqlari bilan ham cheklanadi.

1.1. Muhit va backendni tayyorlash

1-qadam. Tayyor VM bo‘lmasa, VMware yoki VirtualBoxda Ubuntu 24.04 LTS ISO orqali 2 vCPU, 4 GB RAM, 25 GB disk va NAT adapterli VM yarating. OTni o‘rnating, oddiy foydalanuvchi bilan kiring va terminal oching. Quyidagi buyruqlar Ubuntu Bash uchun.

```
sudo apt update
sudo apt install -y apache2 libapache2-mod-security2 openssl curl python3
mkdir -p ~/a9-lab
cd ~/a9-lab
sudo mkdir -p /opt/a9-lab/{run,logs,tls}
sudo chmod 755 /opt/a9-lab /opt/a9-lab/{run,logs,tls}
```

2-qadam. 1-ilovadagi backend.py faylini ~/a9-lab katalogiga saqlang. 2-ilovadagi A va B qismlarni birlashtirib httpd.conf fayli yarating. Oddiy qo‘shtirnoqlar va kod surilishlarini saqlang. Nano ishlatilsa: nano backend.py va nano httpd.conf.

```
python3 -m py_compile backend.py
apache2 -v
openssl version
curl --version
dpkg-query -W apache2 libapache2-mod-security2
python3 -u backend.py 2>&1 | tee backend.log
```

1-terminal ochiq qoladi. A9 backend: http://127.0.0.1:8009 yozuvi chiqadi. -u jurnaldagi yozuvlarning darhol ko‘rinishiga yordam beradi. 2-terminalda tekshiring:

```
cd ~/a9-lab
curl --noproxy "*" -i http://127.0.0.1:8009/
curl --noproxy "*" -i http://127.0.0.1:8009/headers
```

Ikkala javob 200. Birinchida A9-BACKEND, ikkinchida marker, host, proto va client maydonlari bor. To‘g‘ridan to‘g‘ri murojaatda proto/client null bo‘lishi mumkin: ularni keyin proksi kiritadi.

Nega Apache alohida konfiguratsiya bilan boshlanadi?

Ushbu ish /opt/a9-lab/httpd.conf orqali alohida Apache nusxasini ishga tushiradi. Uning PID, log va portlari o‘ziga tegishli. 80/443dagi mavjud saytlar konfiguratsiyasi ishlatilmaydi. Modullar 2-ilovada LoadModule orqali yuklanadi; ushbu nusxa uchun a2ensite yoki a2enmod bajarish talab etilmaydi.

Dalil D1. VM parametrlari, dastur versiyalari, backend 200 javobi va server terminali. Oldingi A8 fayllari bu ishga kerak emas. Apache konfiguratsiyasini hali ishga tushirmang: avval sertifikat yaratiladi.

1.2. O‘quv CA va server sertifikatini yaratish

3-qadam. Quyidagi buyruqlar 2-terminalda bajariladi. ~/a9-ca faqat shu ish sertifikatlari uchun. ca.key — CAning yopiq kaliti, server.key — serverning yopiq kaliti. Ular hisobotga yoki umumiy papkaga qo‘shilmaydi. ca.crt esa mijozga ishonchni tekshirish uchun beriladi.

```
umask 077
mkdir -p ~/a9-ca
cd ~/a9-ca
openssl req -x509 -newkey rsa:2048 -nodes -days 7 \
  -keyout ca.key -out ca.crt -subj "/CN=A9 Lab Root" \
  -addext "basicConstraints=critical,CA:TRUE" \
  -addext "keyUsage=critical,keyCertSign,cRLSign"
openssl req -new -newkey rsa:2048 -nodes \
  -keyout server.key -out server.csr -subj "/CN=a9.test"
cat > server.ext <<'EOF'
basicConstraints=critical,CA:FALSE
keyUsage=critical,digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
subjectAltName=DNS:a9.test
EOF
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key \
  -CAcreateserial -out server.crt -days 7 -sha256 -extfile server.ext
```

-nodes yopiq kalitni parol bilan shifrlamaydi; shu sababli umask 077 va fayl huquqlari muhim. 7 kunlik muddat o‘quv uchun tanlangan. Bu CA brauzer yoki operatsion tizimning global ishonch omboriga qo‘shilmaydi. [6]

Sertifikat mazmuni va nomini tekshirish

```
openssl x509 -in server.crt -noout -subject -issuer -dates
openssl x509 -in server.crt -noout -ext subjectAltName
openssl verify -CAfile ca.crt -verify_hostname a9.test server.crt
openssl verify -CAfile ca.crt -verify_hostname wrong.test server.crt
```

Kutiladigan natija: SAN ichida DNS:a9.test; to‘g‘ri nom uchun server.crt: OK. wrong.test uchun hostname mismatch xatosi. Sertifikat CA tomonidan imzolangan bo‘lishi uning istalgan nomga mosligini anglatmaydi.

```
sudo install -m 600 server.key /opt/a9-lab/tls/server.key
sudo install -m 644 server.crt /opt/a9-lab/tls/server.crt
```

Dalil D2. Subject, issuer, muddat, SAN, to‘g‘ri va noto‘g‘ri nom natijalari. Buyruqlarni qayta bajarish kalit va sertifikatlarni yangilaydi: server nusxalarini ham yangilab, Apache qayta yuklanadi.

1.3. TLS proksisini ishga tushirish

4-qadam. httpd.conf ichidagi yo‘llar /opt/a9-labga mos bo‘lsin. Dastlab SecRuleEngine DetectionOnly holatini saqlang. Alohida nusxa har safar ayni -f yo‘li bilan boshqariladi.

```
cd ~/a9-lab
sudo install -m 644 httpd.conf /opt/a9-lab/httpd.conf
sudo apache2 -t -f /opt/a9-lab/httpd.conf
sudo apache2 -M -f /opt/a9-lab/httpd.conf
sudo apache2 -f /opt/a9-lab/httpd.conf -k start
```

-t natijasi Syntax OK bo‘lishi kerak. -M ro‘yxatida ssl_module, headers_module, proxy_module, proxy_http_module va security2_module ko‘rinsin. Syntax OK chiqmaguncha start yoki graceful bajarilmaydi.

```
ss -ltn | grep -E ":(8009|8089|8449)[[:space:]]"
CA="$HOME/a9-ca/ca.crt"
URL=https://a9.test:8449
curl --noproxy "*" --resolve a9.test:8089:127.0.0.1 \
  -i http://a9.test:8089/
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" -i "$URL/"
```

Tinglovchilar 127.0.0.1da ko‘rinsin. HTTP javobi 301, Location: https://a9.test:8449/. HTTPS javobi 200 va A9-BACKEND. --resolve IPni belgilaydi, ammo SNI, Host va sertifikat nomi tekshiruvida a9.test saqlanadi. --noproxy laboratoriya so‘rovini tashqi proksiga yubormaydi. [2]

Qaysi sozlama qanday vazifa bajaradi?

Sozlama	Vazifasi
SSLProtocol -all +TLSv1.2 +TLSv1.3	Faqat TLS 1.2 va 1.3ni ruxsat etadi
ProxyRequests Off	Forward proxy rejimi o‘chiq
ProxyPass / http://127.0.0.1:8009/	So‘rovni belgilangan backendga uzatadi
RequestHeader set X-Forwarded-Proto	Backend uchun tashqi sxemani https qiladi
ServerTokens Prod; ServerSignature Off	Apache banner tafsilotlari va imzosini kamaytiradi
TraceEnable Off	TRACE usulini o‘chiradi

ServerTokens backendning o‘zi yuboradigan barcha ma’lumotni yashirish kafolati emas. ProxyPassReverse bu backendda maxsus sinovdan o‘tkazilmagan; u umumiy teskari proksi konfiguratsiyasining bir qismi. [3]

Dalil D3. Syntax OK, modullar, 3 port, 301 yo‘naltirish va sertifikat tekshiruvi bilan olingan 200 javobi. curl -k ishlatilmaydi: u CA va nom tekshiruvini chetlab o‘tadi.

1.4. TLS va proksi sarlavhalarini tekshirish

5-topshiriq. Ishonch, nom va protokolni alohida nazorat qiling. CA o‘zgaruvchisi 1.3-bo‘limda belgilangan. Yangi terminal ochilsa uni qayta belgilang.

```
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  -i "$URL/"
curl --noproxy "*" --resolve wrong.test:8449:127.0.0.1 \
  --cacert "$CA" -i https://wrong.test:8449/
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" --tlsv1.2 --tls-max 1.2 -i "$URL/"
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" --tlsv1.3 --tls-max 1.3 -i "$URL/"
```

Sinov	Kutiladigan natija
CA ko‘rsatilmagan	curl (60): ishonch zanjiri tasdiqlanmaydi
CA to‘g‘ri, nom wrong.test	curl (60): sertifikat nomi mos emas
TLS 1.2 aniq tanlangan	200 va A9-BACKEND
TLS 1.3 aniq tanlangan	200 va A9-BACKEND

TLS protokoli bo‘yicha qo‘shimcha dalil uchun quyidagini bajaring. Protocol version va Verification: OK satrlarini qayd eting:

```
openssl s_client -connect 127.0.0.1:8449 -servername a9.test \
  -CAfile "$CA" -verify_hostname a9.test -verify_return_error \
  -tls1_2 -brief < /dev/null
```

Eski TLSni rad etish qo‘shimcha sinovi: shu buyruqda -tls1_2 o‘rniga -tls1 va -cipher "DEFAULT:@SECLEVEL=0" bering. Serverdan protocol version alert kelsa rad etish dalili bo‘ladi. Faqat mijozning no protocols/no ciphers xatosi server rad etganini isbotlamaydi; faol konfiguratsiya va TLS 1.2/1.3 muvaffaqiyati bilan birga talqin qiling.

Mijoz kiritgan proksi sarlavhasiga ishonmaslik

```
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" -H "X-Forwarded-Proto: http" \
  -H "X-Forwarded-For: 198.51.100.77" "$URL/headers"
```

Kutiladigan JSON: proto=https, client=127.0.0.1; host odatda a9.test:8449. Kiritilgan soxta IP backendga ishonchli mijoz manzili sifatida o‘tmasin. Bu faqat shu proksi yo‘li va shu ikki sarlavha tekshiruvidir; bevosita backend so‘rovi proksi nazoratidan o‘tmaydi.

Dalil D4. Ikki xil sertifikat xatosi, TLS 1.2/1.3 natijalari va almashtirilgan proksi sarlavhalari. TLS xatosida HTTP status bo‘lmasligi tabiiy: HTTP bosqichiga yetilmagan.

1.5. Xavfsizlik sarlavhalari va WAF kuzatish rejimi

6-topshiriq. HTTPSning qonuniy va mavjud bo‘lmagan yo‘llariga so‘rov yuboring. -D - javob sarlavhalarini, -o /dev/null esa tanani yashirishni bildiradi. HEAD o‘rniga GET ishlatiladi.

```
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" -D - -o /dev/null "$URL/"
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" -D - -o /dev/null "$URL/missing"
```

Sarlavha	Qiymat / maqsad
X-Content-Type-Options	nosniff; MIME turini o‘zboshimcha talqin qilishni cheklaydi
X-Frame-Options	DENY; sahifani freymda ochishga yo‘l qo‘ymaydi
Referrer-Policy	no-referrer; referrer yuborilmaydi
Content-Security-Policy	default-src 'self'; frame-ancestors 'none'; object-src 'none'

200 va 404 javoblarida sarlavhalar bo‘lsin. Header always error javoblarini ham qamraydi. CSP mazmuni bu sodda sahifaga mos; haqiqiy saytga ko‘r-ko‘rona ko‘chirish resurslarni buzishi mumkin. Bu bosqich brauzer siyosatining amaliy bajarilishini emas, javob sozlamasini tekshiradi. [5, 7]

7-topshiriq. DetectionOnly: moslik bor, bloklash yo‘q

```
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" -i "$URL/?probe=A9-WAF-TEST&case=before"
sudo grep "1009001" /opt/a9-lab/logs/error.log
tail -n 5 ~/a9-lab/backend.log
```

Javob 200 va A9-BACKEND; error.logda id 1009001 va A9 marker; backend.logda case=before so‘rovi ko‘rinishi kerak. DetectionOnly kuzatadi, deny amalini bajarmaydi. Audit jurnalidagi mos tranzaksiya ham dalil bo‘la oladi. [4]

Dalil D5. 200/404 sarlavhalari, marker so‘rovining 200 javobi, qoida IDsi va backendga yetib borgan so‘rov. WAFni “yoqilgan” deb aytishdan oldin rejimini tekshiring: modul yuklangani bloklash faol ekanini bildirmaydi.

HSTS haqida. Bu nostandart 8089/8449 portli asosiy konfiguratsiyada HSTS ataylab berilmagan. HSTS portni har doim 8449ga almashtirmaydi; brauzer 8089ni HTTPS bilan ochishga urinishi mumkin. Haqiqiy 80/443 joylashtirishida ishonchli HTTPSdan keyin HSTS muddatini bosqichma-bosqich oshirish rejalashtiriladi. [8]

1.6. WAFni bloklashga o‘tkazish va qayta sinash

8-topshiriq. Faol konfiguratsiyaning nusxasini oling. Faqat /opt/a9-lab/httpd.conf faylidagi bitta SecRuleEngine DetectionOnly satrini SecRuleEngine On bilan almashtiring. Qoidaning IDsi va mazmuni o‘zgarmasin.

```
sudo cp /opt/a9-lab/httpd.conf /opt/a9-lab/httpd.before.conf
sudo nano /opt/a9-lab/httpd.conf
```

```
SecRuleEngine On
```

```
sudo apache2 -t -f /opt/a9-lab/httpd.conf
sudo apache2 -f /opt/a9-lab/httpd.conf -k graceful
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" -i "$URL/?probe=A9-WAF-TEST&case=after"
sudo grep "1009001" /opt/a9-lab/logs/error.log
tail -n 8 ~/a9-lab/backend.log
```

Kutiladigan natija: 403; error.logdagi ayni yangi so‘rovda id 1009001 va Access denied yozuvi. Backend jurnalida case=after yozuvi paydo bo‘lmasligi kerak. Oldingi case=before yozuvi qolishi tabiiy. URLdagi case parametrlarini va vaqtni farqlab solishtiring.

Qonuniy so‘rovlar va xato javobini saqlash

```
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" -i "$URL/?probe=hello&case=control"
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" -i "$URL/"
curl --noproxy "*" --resolve a9.test:8449:127.0.0.1 \
  --cacert "$CA" -i "$URL/missing"
```

Natijalar: 200, 200, 404. Bloklangan 403 javobida ham xavfsizlik sarlavhalari borligini tekshiring. Xizmatning barcha so‘rovga 403 berishi muvaffaqiyatli himoya hisoblanmaydi.

Qoida va uning cheklovi

ARGS:probe so‘rov parametrini, @streq aynan tenglikni tekshiradi. phase:2 so‘rovni qayta ishlash bosqichini, deny va status:403 bloklashni belgilaydi. t:none qoida uchun qo‘shimcha transformatsiyalarni bekor qiladi. Query parametrlarining standart dekodlanishi baribir bajariladi. [4]

Ixtiyoriy nazorat: probe=A9%2DWAF%2DTEST yuboring; dekodlangach markerga teng bo‘lgani uchun 403 kutiladi. probe=hello 200 bo‘lib qoladi. Bitta marker qoidasi SQLi, XSS, BOLA yoki barcha kodlash usullariga qarshi umumiy himoya emas. OWASP CRSni o‘rnatish va sozlash bu variantning qo‘shimcha loyihasi bo‘lishi mumkin.

Dalil D6. Bir xil qoida bilan DetectionOnlydagi 200 va Ondagi 403; yangi ID/jurnal yozuvi; backendga yetmagan case=after; qonuniy 200lar. Boshqa qoida yoki xizmat sababli kelgan 403ni ayni WAF qoidasining isboti deb qabul qilmang.

1.7. Vaqt rejasi, hisobot va yakunlash

Bosqich	Vaqt	Dalil
Tayyor muhit va backend	8 daqiqa	D1
CA va sertifikat	15 daqiqa	D2
Proksi, TLS va yo‘naltirish	15 daqiqa	D3–D4
Sarlavhalar va DetectionOnly	12 daqiqa	D5
On, log va qonuniy qayta sinov	20 daqiqa	D6
Hisobot va himoya	10 daqiqa	Yakuniy jadval
Jami	80 daqiqa	Tayyorgarlik alohida

Hisobot uchun natijalar jadvali

Sinov	Kutiladigan natija	Amaldagi dalil
Sertifikat: a9.test / wrong.test	OK / hostname mismatch	_____
HTTP 8089	301; HTTPS 8449ga Location	_____
HTTPS + CA + to‘g‘ri nom	200; A9-BACKEND	_____
CA yo‘q / noto‘g‘ri nom	curl 60 / curl 60; sabablari farqli	_____
TLS 1.2 / TLS 1.3	200 / 200	_____
200 / 404 / WAF 403	Xavfsizlik sarlavhalari bor	_____
DetectionOnly + marker	200; log ID; backendga yetadi	_____
On + marker	403; log ID; backendga yetmaydi	_____
On + oddiy so‘rov	200; xizmat ishlaydi	_____

Hisobot: titul (fan, 5-semestr, A9, 1-variant, F.I.Sh., guruh, sana), maqsad, VM va dastur versiyalari, arxitektura, D1–D6 dalillari, natijalar jadvali hamda 5–7 gap xulosa. Har bir dalilda buyruq, vaqt, status va muhim maydon ko‘rinsin. Sertifikat ochiq ma’lumotlari kiritilishi mumkin; yopiq kalitlar kiritilmaydi.

```
sudo apache2 -f /opt/a9-lab/httpd.conf -k stop
```

Backend terminalida Ctrl+C bosing. Hisobot, faol httpd.conf nusxasi va backend.py faylini saqlang. Oddiy apache2 xizmatini systemctl bilan to‘xtatish o‘rniga ayni laboratoriya nusxasini -f yo‘li bilan boshqaring.

Tiklash. Sinovda xato yuz bersa httpd.before.conf nusxasini faol faylga qaytaring, -t bilan tekshiring va -k graceful bajaring. Bu WAFni yana DetectionOnlyga qaytaradi; yakuniy holat sifatida hisobotda On va qayta sinov natijalari talab qilinadi.

1.8. Muammolar, baholash va nazorat savollari

Belgi	Tekshirish / yechim
Syntax error / Invalid command	Ko‘rsatilgan satr, LoadModule yo‘li, qo‘shtirnoq va satr oxiri \ belgisini tekshiring.
Sertifikat yoki key topilmadi	/opt/a9-lab/tls yo‘llari va install bosqichini tekshiring.
curl (60)	CA yo‘li, SAN, nom va VM vaqti; -k bilan muammoni yashirmang.
Connection refused	Alohida Apache ishga tushganmi? 8089/8449 band emasmi? -f yo‘li to‘g‘rimi?
502/503 yoki proxy xatosi	8009 backend ishlayotganini tekshiring; WAF 403 bilan adashtirmang.
On bo‘lsa ham marker 200	Faol fayl, graceful natijasi va rule ID; tahrirlangan nusxa yuklanganmi?
Logda ID bor, lekin 200	DetectionOnly qolmaganmi? Yangi tranzaksiyani eski yozuvdan ajrating.
SSL session cache warning	Bu minimal konfiguratsiyada session cache yo‘q; ogohlantirish Syntax OKni inkor etmaydi.

Baholash mezonlari

Mezon	Ball
Muhit, backend va arxitektura	1
CA, SAN, sertifikat zanjiri va nom sinovlari	2
TLS, yo‘naltirish va proksi sarlavhalari	2
200/404/403 xavfsizlik sarlavhalari	1
WAF rejimlari, ID, backend va qonuniy qayta sinov	3
Hisobot va nazorat savollariga javob	1
Jami	10

Nazorat savollari

8. CAga ishonish va sertifikat nomi mosligi nega alohida tekshiriladi?
9. TLS 1.2ning ishlashi TLS 1.0 o‘chirilganini o‘zi isbotlaydimi?
10. DetectionOnly va On orasidagi farq qaysi uch dalil bilan ko‘rsatildi?
11. Header always nega error javoblari uchun muhim?
12. Backendga bevosita kirish ochiq bo‘lsa WAF himoyasining chegarasi qanday?
13. Marker qoidasidan to‘liq SQLi/XSS himoyasi haqida xulosa chiqarish nega noto‘g‘ri?

Qo‘shimcha tahlil va rasmiy manbalar

Qo‘shimcha topshiriq. Shu arxitekturani Nginx yoki IISga ko‘chirishda server sertifikati, teskari proksi, xavfsizlik sarlavhalari va WAF integratsiyasi qaysi alohida komponentlarda sozlanishini yozing. Apache direktivalarini boshqa server konfiguratsiyasiga aynan ko‘chirib qo‘yish mumkin emas. Asosiy amaliy natija faqat Apache variantidan olinadi.

Ishlab chiqarish loyihasi uchun: muntazam yangilash, sertifikatni avtomatik yangilash, kalit huquqlari, TLS siyosati, ishonchli proksi chegarasi va WAF qoidalarining yolg‘on musbat natijalarini boshqarish rejalashtiriladi. WAF ilova kodidagi zaifliklarni tuzatish o‘rnini bosmaydi.

[1] Apache — mod_ssl: sertifikat va SSLProtocol

[2] curl — SSL CA Certificates

[3] Apache — mod_proxy: ProxyPass va ProxyPassReverse

[4] ModSecurity v2 — Reference Manual

[5] OWASP — HTTP Headers Cheat Sheet

[6] OpenSSL 3.0 — openssl req

[7] Apache — mod_headers: Header always va RequestHeader

[8] RFC 6797 — HSTS, 8.3: URI Loading and Port Mapping

Manbalar tekshirilgan sana: 07.09.2026. Mavzu foydalanuvchi taqdim etgan fan dasturining A9 bandiga mos. Internetdagi xizmatlarga sinov so‘rovlari yuborish talab qilinmaydi.

O‘qituvchi uchun tekshiruv va tarqatish

14. VMda paketlar o‘rnatilganini tekshiring. ModSecurity va Apache versiyalarini yozib oling; distributiv xavfsizlik yangilanishlarini saqlang.
15. Ilovalardagi fayllar UTF-8 formatida saqlansin. 2-ilovaning A va B qismlari bitta httpd.conf fayliga ketma-ket qo‘shiladi.
16. Har bir talaba o‘z o‘quv CA va server kalitini yaratsin. Umumiy VM nusxasiga oldingi guruhning kalitlari yoki hisobotlari qo‘shilmasin.
17. Darsdan oldin tayyor VMda 200 → 403 → qonuniy 200 zanjirini, ID 1009001ni va backend jurnalini tekshiring.

Tayyorlashdagi tekshiruv chegarasi. Apache konfiguratsiyasi sintaksisi hamda sertifikatning to‘g‘ri/noto‘g‘ri nom bilan tekshirilishi tasdiqlandi. Hujjat tayyorlangan muhitda servis akkauntiga o‘tish cheklangani sababli to‘liq Apache–WAF HTTP sinovi bajarilmadi. Ko‘rsatilgan WAF natijalari darsdan oldin Ubuntu VMda tasdiqlanishi kerak.

1-ilova. backend.py

Faylni ~/a9-lab/backend.py nomi bilan saqlang. Bu sodda backend query parametrlarini bajarmaydi; marker sinovi WAFning ishini kuzatish uchun.

```
import json
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlsplit

class App(BaseHTTPRequestHandler):
    def do_GET(self):
        path = urlsplit(self.path).path
        status, mime = 200, 'text/html; charset=utf-8'
        if path == '/headers':
            mime = 'application/json'
            body = json.dumps({
                'marker': 'A9-BACKEND',
                'host': self.headers.get('Host'),
                'proto': self.headers.get('X-Forwarded-Proto'),
                'client': self.headers.get('X-Forwarded-For'),
            }).encode()
        elif path == '/':
            body = b'<!doctype html><title>A9</title><h1>A9-BACKEND</h1>'
        else:
            status, mime, body = 404, 'text/plain', b'A9-NOT-FOUND'
        self.send_response(status)
        self.send_header('Content-Type', mime)
        self.send_header('Content-Length', str(len(body)))
        self.end_headers()
        self.wfile.write(body)

if __name__ == '__main__':
    print('A9 backend: http://127.0.0.1:8009', flush=True)
    HTTPServer(('127.0.0.1', 8009), App).serve_forever()
```

Jurnalni o‘qish

BaseHTTPRequestHandler har bir kelgan GETni terminalga yozadi. 1.1-bo‘limdagi tee bu chiqishni backend.logga ham saqlaydi. DetectionOnlyda case=before keladi; On holatida WAF bloklagan case=after kelmasligi kerak. Bir nechta testni farqlash uchun vaqt va case qiymatidan foydalaning.

Backend faqat GET uchun yozilgan. curl -I HEAD yuboradi va bu kodda 501ga olib kelishi mumkin; yo‘riqnomadagi -D - bilan GET usulini ishlating.

2-ilova. httpd.conf — A qism

A va B qismlarni ketma-ket bitta httpd.conf fayliga birlashtiring.

```
ServerRoot "/opt/a9-lab"
DefaultRuntimeDir "/opt/a9-lab/run"
PidFile "/opt/a9-lab/run/httpd.pid"
ServerName a9.test
Listen 127.0.0.1:8089
Listen 127.0.0.1:8449
LoadModule mpm_event_module /usr/lib/apache2/modules/mod_mpm_event.so
LoadModule authz_core_module /usr/lib/apache2/modules/mod_authz_core.so
LoadModule authz_host_module /usr/lib/apache2/modules/mod_authz_host.so
LoadModule alias_module /usr/lib/apache2/modules/mod_alias.so
LoadModule ssl_module /usr/lib/apache2/modules/mod_ssl.so
LoadModule headers_module /usr/lib/apache2/modules/mod_headers.so
LoadModule proxy_module /usr/lib/apache2/modules/mod_proxy.so
LoadModule proxy_http_module /usr/lib/apache2/modules/mod_proxy_http.so
LoadModule unique_id_module /usr/lib/apache2/modules/mod_unique_id.so
LoadModule security2_module /usr/lib/apache2/modules/mod_security2.so
User www-data
Group www-data
ServerTokens Prod
ServerSignature Off
TraceEnable Off
ProxyRequests Off
ErrorLog "/opt/a9-lab/logs/error.log"
LogLevel warn
LogFormat "%h %t \"%r\" %>s %b" a9
CustomLog "/opt/a9-lab/logs/access.log" a9
<Directory />
    Require all denied
</Directory>
<Proxy "http://127.0.0.1:8009/">
    Require all granted
</Proxy>
<VirtualHost 127.0.0.1:8089>
    ServerName a9.test
    Redirect permanent / https://a9.test:8449/
</VirtualHost>
```

2-ilova. httpd.conf — B qism

A va B qismlarni ketma-ket bitta httpd.conf fayliga birlashtiring.

```
<VirtualHost 127.0.0.1:8449>
    ServerName a9.test
    SSLEngine On
    SSLCertificateFile "/opt/a9-lab/tls/server.crt"
    SSLCertificateKeyFile "/opt/a9-lab/tls/server.key"
    SSLProtocol -all +TLSv1.2 +TLSv1.3
    ProxyPreserveHost On
    RequestHeader unset X-Forwarded-For
    RequestHeader set X-Forwarded-Proto "https"
    ProxyPass / http://127.0.0.1:8009/ connectiontimeout=3 timeout=5
    ProxyPassReverse / http://127.0.0.1:8009/
    Header always set X-Content-Type-Options "nosniff"
    Header always set X-Frame-Options "DENY"
    Header always set Referrer-Policy "no-referrer"
    Header always set Content-Security-Policy \
        "default-src 'self'; frame-ancestors 'none'; object-src 'none'"
    SecRuleEngine DetectionOnly
    SecRequestBodyAccess On
    SecAuditEngine RelevantOnly
    SecAuditLogType Serial
    SecAuditLogParts ABFHZ
    SecAuditLog "/opt/a9-lab/logs/audit.log"
    SecRule ARGS:probe "@streq A9-WAF-TEST" \
        "id:1009001,phase:2,t:none,deny,status:403,log,auditlog,msg:'A9 marker'"
</VirtualHost>
```

Dastlabki holat DetectionOnly. 1.6-bo‘limda faqat SecRuleEngine On qilib o‘zgartiriladi. Ushbu alohida konfiguratsiya tizimning asosiy apache2.conf, sites-enabled yoki CRS qoidalarini avtomatik yuklamaydi.